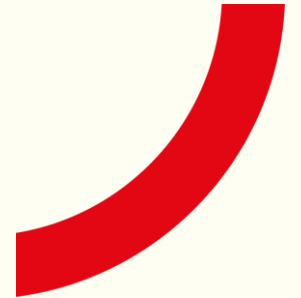


# CSC1029 OO Programming

## Lecture 23 - Insertion Sort



# What's to come in today's lecture?

- Insertion Sorting
- An Informal Introduction to Testing

# Insertion Sort

This algorithm starts with an empty array, inserting elements in their correct location as they are presented.

```
if value is first to be inserted
    insert value at position 0
    return
```

```
find insert position for value
```

```
shuffle values ahead of position,
    (if required)
```

```
insert the value
```

Start:

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
|   |   |   |   |   |   |   |

Insert: 6

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 6 |   |   |   |   |   |   |

Insert: 9

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 6 | 9 |   |   |   |   |   |

Insert: 7

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 6 | 7 | 9 |   |   |   |   |

Insert: 3

| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| 3 | 6 | 7 | 9 |   |   |   |

# Insertion Sort

This algorithm starts with an empty array, inserting elements in their correct location as they are presented.

```
if value is first to be inserted
    insert value at position 0
    return
```

```
find insert position for value
```

```
shuffle values ahead of position,
    (if required)
```

```
insert the value
```

```
public static void insert(int value, int
                           data[], int count) {
    if (count == 0) {
        data[0] = value;
        return;
    }
    // find insert position
    int index, pos;
    for(index=0; index<count; index++) {
        if (data[index]>value) {
            break;
        }
    }
    pos = index; // mark position
    // shuffle values if required
    for(index = count; index>pos; index--) {
        data[index] = data[index-1];
    }
    data[pos] = value; // insert value
}
```

# Insertion Sort – Suitable Test Cases?

- First value to be inserted?



data



# Insertion Sort – Test Case 1

---

```
private static void testCase1() {  
    final int SIZE=7;  
    int data[] = new int[SIZE];  
    System.out.println("+++ Test Case 1: Insert    first element. +++");  
    System.out.print("Before : ");  
    display(data,0);  
    insert(6, data, 0);  
    System.out.print("After  : ");  
    display(data,1);  
    System.out.println("+++ End Test Case ++++++\n");  
}
```

---

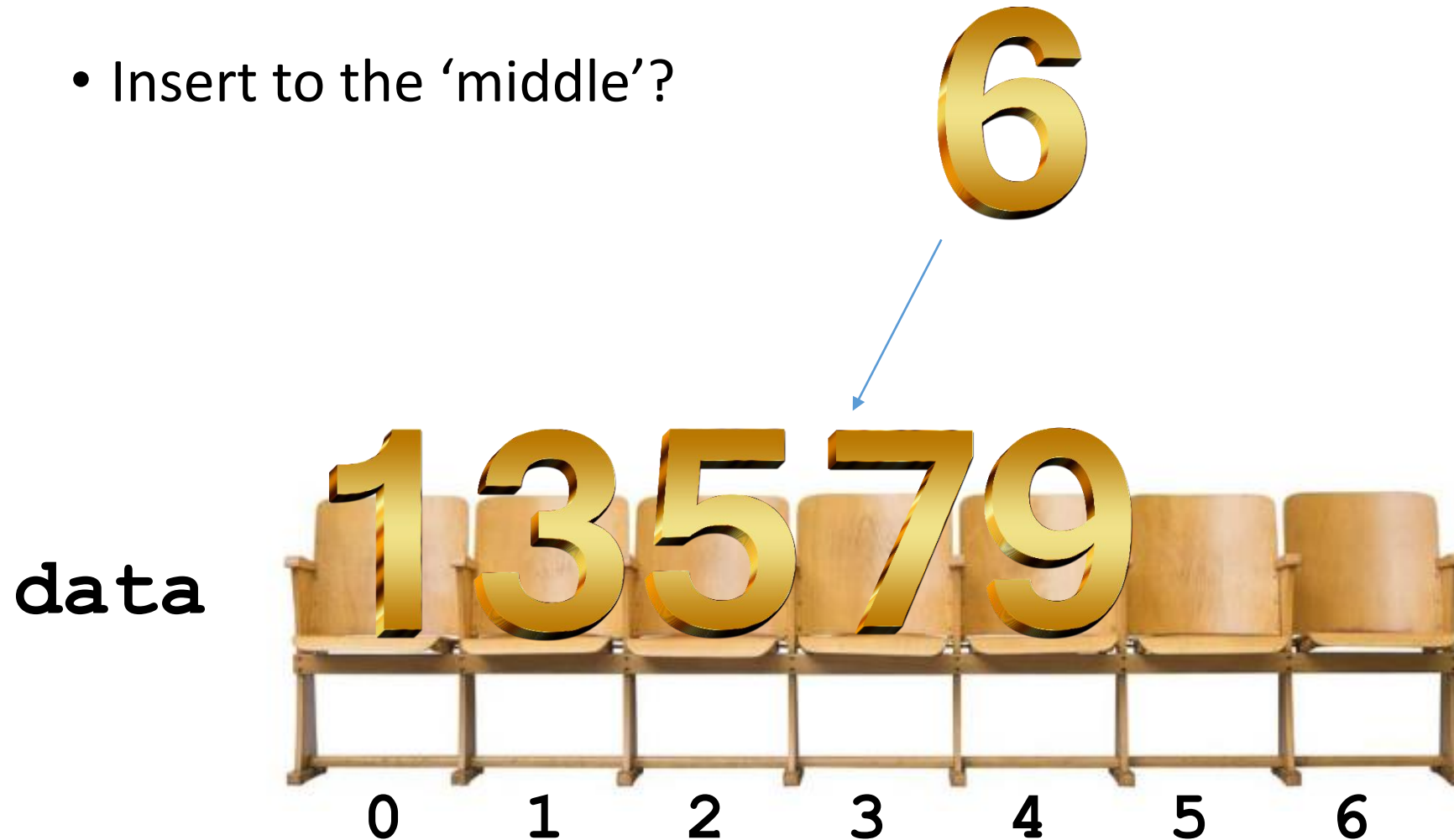
```
+++ Test Case 1: Insert first element. +++  
Before : No data to display.  
After  : [6]  
+++ End Test Case ++++++
```



# Insertion Sort – Suitable Test Cases?

---

- Insert to the 'middle'?



# Insertion Sort – Test Case 2

---

```
private static void testCase2() {  
    int data[] = {1,3,5,7,9,0,0};  
    System.out.println("+++ Test Case 2: Insert in the middle. +++");  
    System.out.print("Before : ");  
    display(data, 5);  
    insert(6, data, 5);  
    System.out.print("After  : ");  
    display(data, 6);  
    System.out.println("+++ End Test Case ++++++\n");  
}
```

---

```
+++ Test Case 2: Insert in the middle. +++  
Before : [1, 3, 5, 7, 9]  
After  : [1, 3, 5, 6, 7, 9]  
+++ End Test Case ++++++
```





# Insertion Sort – Suitable Test Cases?

---

- Insert before first?



# Insertion Sort – Test Case 3

---

```
private static void testCase3() {  
    int data[] = {1,3,5,7,9,0,0};  
    System.out.println("+++ Test Case 3: Insert before first. +++");  
    System.out.print("Before : ");  
    display(data, 5);  
    insert(0, data, 5);  
    System.out.print("After  : ");  
    display(data, 6);  
    System.out.println("+++ End Test Case ++++++\n");  
}
```

---

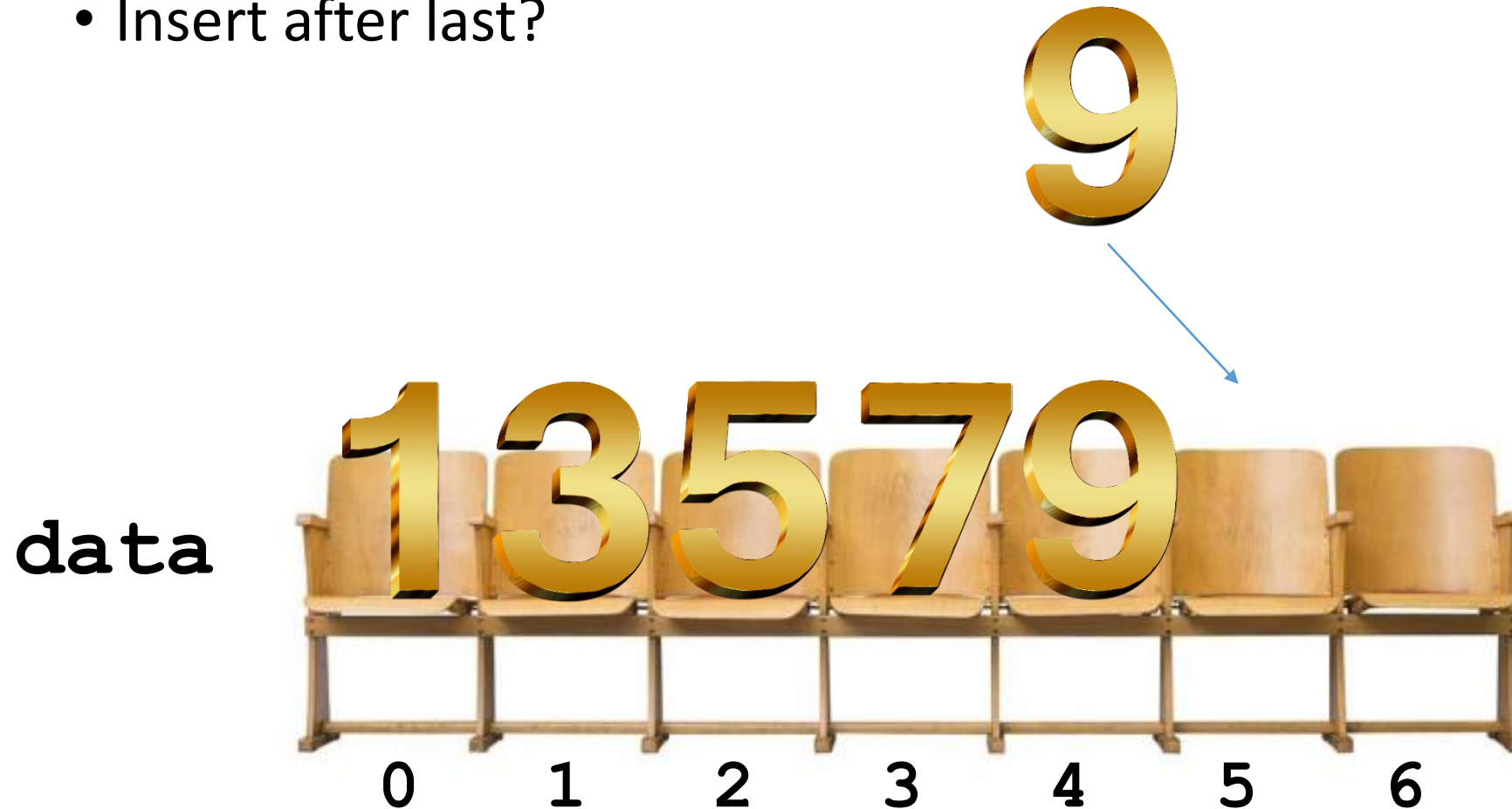
```
+++ Test Case 3: Insert before first. +++  
Before : [1, 3, 5, 7, 9]  
After  : [0, 1, 3, 5, 7, 9]  
+++ End Test Case ++++++
```



# Insertion Sort – Suitable Test Cases?

---

- Insert after last?



# Insertion Sort – Test Case 4

---

```
private static void testCase4() {  
    int data[] = {1,3,5,7,9,0,0};  
    System.out.println("+++ Test Case 4: Insert after last. +++");  
    System.out.print("Before : ");  
    display(data, 5);  
    insert(9, data, 5);  
    System.out.print("After  : ");  
    display(data, 6);  
    System.out.println("+++ End Test Case +++++\n");  
}
```

---

```
+++ Test Case 4: Insert after last. +++  
Before : [1, 3, 5, 7, 9]  
After  : [1, 3, 5, 7, 9, 9]  
+++ End Test Case +++++
```



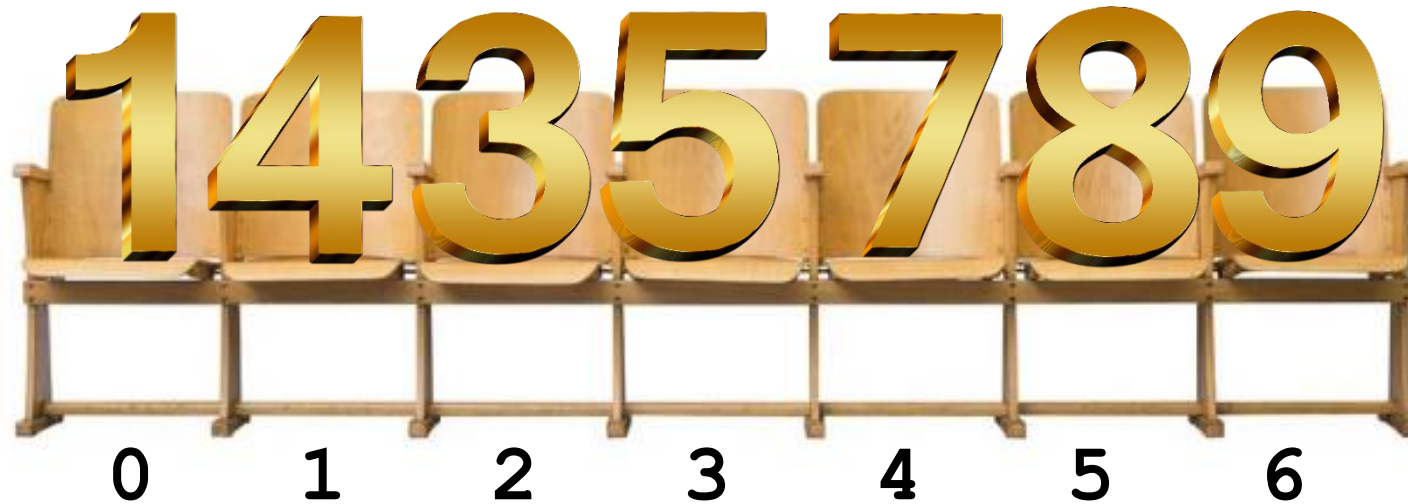
# Insertion Sort – Suitable Test Cases?

---

- What about space?

6 ?

data



# Insertion Sort – Test Case 5

---

```
private static void testCase5() {  
    int data[] = {1,4,3,5,7,8,9};  
    System.out.println("+++ Test Case 5: Insert to full array. +++");  
    System.out.print("Before : ");  
    display(data, 7);  
    insert(6, data, 7);  
    System.out.print("After  : ");  
    display(data, 7);  
    System.out.println("+++ End Test Case ++++++\n");  
}
```

---

```
+++ Test Case 5: Insert to full array. +++  
Before : [1, 4, 3, 5, 7, 8, 9]  
After  : [1, 4, 3, 5, 7, 8, 9]  
+++ End Test Case ++++++
```



# Insertion Sort – Further Code Examples

---





# What's Next?

Sorting Data

Bubble Sort

Software Testing